| | |
|---|---|
| **Course Title**: Cryptography and Network Security Lab | **Course Code**: BCSE309P |
| **Faculty**: Prof. Karthika V | **Slot:** L43 + L44 |
| **Regno:** 23BCE1145 | **Name:** S Vishwajith |

# EX2A – S-DES

## Code:

### Sender:

```
import pickle

import socket


def circular_left_shift(s, n):

    n = n % len(s)

    return s[n:len(s)] + s[0:n]


def get_intermediate_keys(key, P10, P8):

    key_new = ""

    for i in P10:

        key_new += key[i-1]

    key_new = circular_left_shift(key_new[:5], 1) + circular_left_shift(key_new[5:], 1)

    k1, k2 = "", ""

    for i in P8:

        k1 += key_new[i-1]

    key_new = circular_left_shift(key_new[:5], 2) + circular_left_shift(key_new[5:], 2)

    for i in P8:

        k2 += key_new[i-1]

    return [k1, k2]


def S_box(l, r):

    S_0 = [["01", "00", "11", "10"],
```

```
        ["11", "10", "01", "00"],
        ["00", "10", "01", "11"],
        ["11", "01", "11", "01"]]
    S_1 = [["00", "01", "10", "11"],
        ["10", "00", "01", "11"],
        ["11", "00", "01", "00"],
        ["10", "01", "00", "11"]]
    return S_0[int(l[0] + l[3], 2)][int(l[1:3], 2)] + S_1[int(r[0] + r[3], 2)][int(r[1:3], 2)]

def exclusive_or(s1, s2):
    res = ""
    for i in range(len(s1)):
        res += str(int(s1[i]) ^ int(s2[i]))
    return res

def work(PT, EP, k_i, P4):
    l_0, r_0 = PT[:4], PT[4:]
    temp = ""
    for i in EP:
        temp += r_0[i-1]
    temp = exclusive_or(temp, k_i)
    temp = S_box(temp[:4], temp[4:])
    l_1 = ""
    for i in P4:
        l_1 += temp[i-1]
    l = exclusive_or(l_0, l_1)
    return l + r_0

def encrypt(PT, key, P10, P8, P4, IP, IP_inv, EP):
    PT_new = ""
    for i in IP:
```

```
        PT_new += PT[i-1]
    k1, k2 = get_intermediate_keys(key, P10, P8)
    print(f"\nThe subkeys are: {[k1, k2]}.")
    print(f"The plain text after initial permutation: {PT_new}.")
    PT_new = work(PT_new, EP, k1, P4)
    PT_new = PT_new[4:] + PT_new[:4]
    print(f"The plain text after Round 1: {PT_new}.")
    PT_new = work(PT_new, EP, k2, P4)
    print(f"The plain text after Round 2: {PT_new}.")
    CT = ""
    for i in IP_inv:
        CT += PT_new[i-1]
    return CT

def get_inverse_IP(IP):
    IP_inv = []
    for i in range(len(IP)):
        IP_inv.append(IP.index(i+1)+1)
    return IP_inv
print("S-DES sender initialized.\n")
PT = input("Enter the 8-bit plain text in binary: ")
key = input("Enter the 10-bit key in binary: ")
P10 = input("Enter the permutation for 10 bits: ").split(" ")
P8 = input("Enter the permutation for 8 bits: ").split(" ")
P4 = input("Enter the permutation for 4 bits: ").split(" ")
P10 = [int(i) for i in P10]
P8 = [int(i) for i in P8]
P4 = [int(i) for i in P4]
IP = input("Enter the initial permutation for 8 bits: ").split(" ")
IP = [int(i) for i in IP]
IP_inv = get_inverse_IP(IP)
```

```
EP = input("Enter the expansion permutation for 4 to 8 bits: ").split(" ")
EP = [int(i) for i in EP]

CT = encrypt(PT, key, P10, P8, P4, IP, IP_inv, EP)
print(f"The cipher text is: {CT}.")

s = socket.socket()
s.connect(('127.0.0.1', 5995))
s.send(CT.encode().ljust(1024, b'\0'))
s.send(key.encode().ljust(1024, b'\0'))
s.send(pickle.dumps(P10).ljust(1024, b'\0'))
s.send(pickle.dumps(P8).ljust(1024, b'\0'))
s.send(pickle.dumps(P4).ljust(1024, b'\0'))
s.send(pickle.dumps(IP).ljust(1024, b'\0'))
s.send(pickle.dumps(EP).ljust(1024, b'\0'))
print("\nThe Cipher text, key and the permutations have been sent successfully to the receiver.")
s.close()
```

## Receiver:

```
import pickle
import socket

def circular_left_shift(s, n):
    n = n % len(s)
    return s[n:len(s)] + s[0:n]

def get_intermediate_keys(key, P10, P8):
    key_new = ""
    for i in P10:
        key_new += key[i-1]
    key_new = circular_left_shift(key_new[:5], 1) + circular_left_shift(key_new[5:], 1)
    k1, k2 = "", ""
```

```
    for i in P8:
        k1 += key_new[i-1]
    key_new = circular_left_shift(key_new[:5], 2) + circular_left_shift(key_new[5:], 2)
    for i in P8:
        k2 += key_new[i-1]
    return [k1, k2]

def S_box(l, r):
    S_0 = [["01", "00", "11", "10"],
           ["11", "10", "01", "00"],
           ["00", "10", "01", "11"],
           ["11", "01", "11", "01"]]
    S_1 = [["00", "01", "10", "11"],
           ["10", "00", "01", "11"],
           ["11", "00", "01", "00"],
           ["10", "01", "00", "11"]]
    return S_0[int(l[0] + l[3], 2)][int(l[1:3], 2)] + S_1[int(r[0] + r[3], 2)][int(r[1:3], 2)]

def exclusive_or(s1, s2):
    res = ""
    for i in range(len(s1)):
        res += str(int(s1[i]) ^ int(s2[i]))
    return res

def work(PT, EP, k_i, P4):
    l_0, r_0 = PT[:4], PT[4:]
    temp = ""
    for i in EP:
        temp += r_0[i-1]
    temp = exclusive_or(temp, k_i)
    temp = S_box(temp[:4], temp[4:])
```

```
    l_1 = ""
    for i in P4:
        l_1 += temp[i-1]
    l = exclusive_or(l_0, l_1)
    return l + r_0

def decrypt(CT, key, P10, P8, P4, IP, IP_inv, EP):
    CT_new = ""
    for i in IP:
        CT_new += CT[i-1]
    k2, k1 = get_intermediate_keys(key, P10, P8)
    print(f"\nThe subkeys are: {[k1, k2]}.")
    print(f"The cipher text after initial permutation: {CT_new}.")
    CT_new = work(CT_new, EP, k1, P4)
    CT_new = CT_new[4:] + CT_new[:4]
    print(f"The cipher text after Round 1: {CT_new}.")
    CT_new = work(CT_new, EP, k2, P4)
    print(f"The cipher text after Round 2: {CT_new}.")
    PT = ""
    for i in IP_inv:
        PT += CT_new[i-1]
    return PT

def get_inverse_IP(IP):
    IP_inv = []
    for i in range(len(IP)):
        IP_inv.append(IP.index(i+1)+1)
    return IP_inv

server = socket.socket()
server.bind(('127.0.0.1', 5995))
```

```
server.listen(1)
print("S-DES Receiver initialized and listening for sender.")
conn, addr = server.accept()
print(f"Connection established with sender at {addr}.")

CT = conn.recv(1024).decode().rstrip('\0')
print(f"The received cipher text is: {CT}.")
key = conn.recv(1024).decode().rstrip('\0')
P10 = pickle.loads(conn.recv(1024).rstrip(b'\0'))
P8 = pickle.loads(conn.recv(1024).rstrip(b'\0'))
P4 = pickle.loads(conn.recv(1024).rstrip(b'\0'))
IP = pickle.loads(conn.recv(1024).rstrip(b'\0'))
IP_inv = get_inverse_IP(IP)
EP = pickle.loads(conn.recv(1024).rstrip(b'\0'))

conn.close()
server.close()

PT = decrypt(CT, key, P10, P8, P4, IP, IP_inv, EP)
print(f"The plain text is: {PT}.")
```

# Output:

## Sender:

```
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-4 (S-DES)> python .\sender.py
S-DES sender initialized.

Enter the 8-bit plain text in binary: 10010111
Enter the 10-bit key in binary: 1010000010
Enter the permutation for 10 bits: 3 5 2 7 4 10 1 9 8 6
Enter the permutation for 8 bits: 6 3 7 4 8 5 10 9
Enter the permutation for 4 bits: 2 4 3 1
Enter the initial permutation for 8 bits: 2 6 3 1 4 8 5 7
Enter the expansion permutation for 4 to 8 bits: 4 1 2 3 2 3 4 1

The subkeys are: ['10100100', '01000011'].
The plain text after initial permutation: 01011101.
The plain text after Round 1: 11011010.
The plain text after Round 2: 00101010.
The cipher text is: 00111000.

The Cipher text, key and the permutations have been sent successfully to the receiver.
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-4 (S-DES)>
```

## Receiver:

```
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-4 (S-DES)> python .\receiver.py
S-DES Receiver initialized and listening for sender.
Connection established with sender at ('127.0.0.1', 64210).
The received cipher text is: 00111000.

The subkeys are: ['01000011', '10100100'].
The cipher text after initial permutation: 00101010.
The cipher text after Round 1: 10101101.
The cipher text after Round 2: 01011101.
The plain text is: 10010111.
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-4 (S-DES)>
```